# Предиктивная аналитика

## Лекция 5. Специализированные модели

Амелия Вадимовна Полей-Добронравова
18.03.2026

# Prophet

# Prophet

Библиотека от Facebook для прогнозирования на основе аддитивной модели разложения

- **Автоматически определяет сезонные компоненты**, которые могут быть как ежегодными, так и еженедельными. Вы также можете добавлять собственные сезонные эффекты
- Способен **справляться с пропусками** в данных
- Вы можете настраивать количество сезонных циклов, влияние праздников, и тд

# Prophet $y(t) = g(t) + s(t) + h(t) + \epsilon_t$

## Кусочно-линейный тренд

$$g(t) = k + \sum_{j=1}^{m} \delta_j \cdot \mathbb{I}(t > \tau_j) \cdot (t - \tau_j) + b$$

- k — базовая скорость роста
- τj — точки изменения (changepoints)
- δj — изменение наклона в точке τj
- m — число потенциальных changepoints (по умолчанию 25)

$$\mathbb{I}(A) = \begin{cases} 1, & \text{если утверждение } A \text{ истинно} \\ 0, & \text{если утверждение } A \text{ ложно} \end{cases}$$

## Логистический тренд

$$g(t) = \frac{C(t)}{1 + \exp\left(-\left(k + \sum \delta_j \cdot \mathbb{I}(t > \tau_j)\right) \cdot \left(t - \left(m + \sum \gamma_j\right)\right)\right)}$$

- C(t) — вместимость (capacity), может быть константой или функцией
- γj — коррекция сдвига

При увеличении показателя снижается темп его роста. **Типичный пример:** рост аудитории приложения

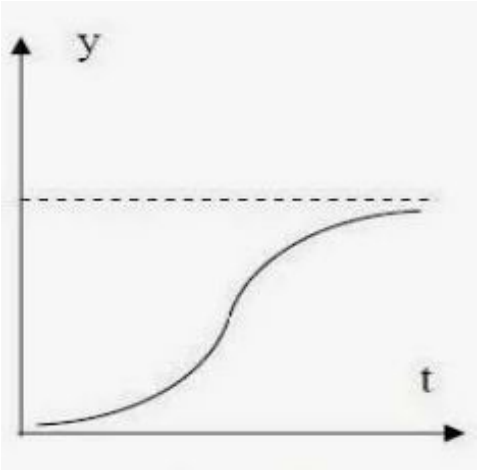
Библиотека умеет по историческим данным выбирать оптимальные точки изменения тренда и принимать их заданные вручную. Например, если известны даты релизов новой функциональности

# Prophet

<u>Параметр непрерывности γj</u>

Если мы просто изменим скорость k в точке τj, график тренда «разорвется» (появится скачок). Чтобы кривая оставалась гладкой, нужно сдвинуть ось времени

$$\gamma_j = \left(\tau_j - m - \sum_{l=1}^{j-1} \gamma_l\right) \cdot \left(1 - \frac{k + \sum_{l=1}^{j-1} \delta_l}{k + \sum_{l=1}^{j} \delta_l}\right)$$




1. Мы меняем наклон кривой в точке $\tau j$
2. Чтобы новая наклонная линия прошла через ту же точку $\tau j$, что и старая, нам нужно сдвинуть «центр» сигмоиды.
3. $\gamma j$ — это величина этого сдвига. Он гарантирует, что $g(t)$ непрерывна в каждой точке $\tau j$

# Prophet $y(t) = g(t) + s(t) + h(t) + \epsilon_t$

## s(t) - сезонные компоненты через ряд Фурье

$$s(t) = \sum_{n=1}^{N} \left( a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right)$$

- P — период
- n — номер гармоники, «Базовая волна» (n=1), «волна в 2 раза быстрее» (n=2), тп
- N — порядок Фурье, сколько разных «скоростей волн» мы используем
- an, bn — коэффициенты силы волны и где ее пик

В тренировочных данных должно быть хотя бы 2 периода сезонности для настройки

**Чем сложнее паттерн внутри периода, тем больше порядок Фурье**

Но: больше порядок → больше риск переобучения

Недельная: начните с 3, увеличивайте до 5, если видите неучтённые пики (например, «супер-пятница»).

Годовая: начните с 10, уменьшайте до 5, если кривая слишком «волнистая» и неинтерпретируемая

# Prophet

Есть возможность указать **мультипликативную сезонность**

$$y(t) = g(t) \cdot (1 + s(t) + h(t)) + \varepsilon_t$$

Формула самой сезонности не меняется

$$s(t) = \sum_{n=1}^{N} \left( a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right)$$

но an, bn  теперь — доли от тренда

**Данные:** продажи от 500 до 2000 единиц

**Аддитивная сезонность**

$a_1$ = 150 → "пятница добавляет 150 продаж"

**Мультипликативная сезонность**

$a_1$ = 0.15 → "пятница добавляет 15% к текущему уровню"

# Prophet $y(t) = g(t) + s(t) + h(t) + \epsilon_t$

## h(t) - аномальные дни

$$h(t) = \sum_{i=1}^{L} \kappa_i \cdot \mathbb{I}(t \in [D_i - l_i, D_i + u_i])$$

- D i — дата события
- $[li, ui]$ — окно влияния (дни до/после)
- $\kappa i$ ~ $N$(0,$\nu$^2) — эффект события с априорным масштабом $\nu$

## Внешние регрессоры

$$y(t) = g(t) + s(t) + h(t) + \sum_{j=1}^{R} \beta_j x_j(t) + \varepsilon_t$$

- xj(t) — внешний регрессор (например, температура, цена)
- βj — оцениваемый коэффициент

# Prophet

Под капотом язык вероятностного программирования **Stan** для построения **байесовской кривой**

**Шаг обучения 1**

Нормализация: Время t приводится к диапазону [0,1], целевая переменная y масштабируется

Выбор точек изменения тренда: По умолчанию берётся 25 потенциальных точек ($\tau_j$)

Они равномерно распределяются по первым 80% истории, чтобы модель не пыталась подстроиться под шум в самом конце данных

# Prophet

**Шаг обучения 2. Оптимизация**

В байесовском подходе минимизация функции потерь это

Найти такие θ, чтобы максимизировать P(θ | Data),
где θ — все параметры

**Шаг обучения 3. Оценка неопределенности**

По умолчанию быстрый MAP (Maximum A Posteriori) на основе остатков (шума) и неопределённости тренда

Можно включить MCMC (Markov Chain Monte Carlo), он генерирует 1000+ вариантов кривой методом Монте-Карло

Важна регуляризация априорных распределений, штрафуя за сложность через уменьшение параметра prior_scale

# Prophet vs SARIMA

SARIMA-модели требуют больше усилий:

- нужно привести ряд к стационарному
- подобрать начальные приближения
- потратить немало времени на подбор гипер-параметров алгоритма
- будут проблемы с пропусками в данных
- поддерживают только одну сезонность, а в prophet мультисезонность
- низкая интерпретация для бизнеса
- хороши только на краткосрочном прогнозе

Для 80% задач prophet подойдет больше

В 20% вам важна автокорреляция больше декомпозиции ряда

# Где нет смысла в Prophet

**Данные без четкой сезонности или тренда**

- Высокочастотные данные (например, данные о сделках на фондовом рынке), где изменения часто зависят от случайных факторов.
- Временные ряды без тренда и сезонности

**Короткие временные ряды**

**Сложные временные ряды с несколькими иерархическими уровнями**, где требуется моделирование взаимодействий между различными уровнями данных (например, временные ряды с разными регионами или продуктами)

**Временные ряды с быстрыми, непредсказуемыми изменениями**:
Для таких данных могут подойти более гибкие модели, такие как LSTM

Практика

# Модели экспоненциального сглаживания (ETS)

# ETS - Error Trend Seasonality

Основываются на концепции экспоненциального сглаживания
Прогноз строится как взвешенная сумма прошлых наблюдений, где веса убывают экспоненциально по мере удаления в прошлое
*Не требуют стационарность данных*

**Формула прогноза для аддитивной модели**

$$\hat{y}_{t+h|t} = l_t + h \cdot b_t + s_{t+h-m(k+1)}$$

*Где $h$ — горизонт прогноза, $m$ — период сезонности.*

l - **уровень**, экспоненциально взвешенное среднее. Показывает сглаженное базовое значение ряда на каждом шаге. Эта компонента выступает основой, вокруг которой колеблется временной ряд. В отличие от простого среднего, уровень постоянно обновляется, отражая текущее положение данных с учётом недавней динамики
b - **тренд**
s - **сезонность**

# Уравнения перехода

## Уровень

$$l_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(l_{t-1} + b_{t-1})$$

($y_t - s_{t-m}$) — очищаем текущее наблюдение от сезонности

($l_{t-1} + b_{t-1}$) — наш прогноз на момент t, сделанный ранее

α — взвешиваем новую информацию против старого прогноза

## Тренд

$$b_t = \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1}$$

($l_t - l_{t-1}$) — фактическое изменение уровня за шаг

$b_{t-1}$ — предыдущая оценка тренда

β — насколько доверять последнему изменению уровня

## Сезонность

$$s_t = \gamma(y_t - l_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m}$$

($y_t - l_{t-1} - b_{t-1}$) — фактическая сезонность в момент t (наблюдение минус тренд)

$s_{t-m}$ — сезонность из предыдущего цикла (например, январь прошлого года)

γ — как быстро обновлять сезонный паттерн

# Типы Моделей ETS

- **ETS(AAM)**: Аддитивная ошибка, аддитивный тренд, мультипликативная сезонность

  $$(l_t + h \cdot b_t) \cdot s_{t+h-m(k+1)}$$

- **ETS(AMA)**: Аддитивная ошибка, мультипликативный тренд, аддитивная сезонность

  $$l_t \cdot b_t^h + s_{t+h-m(k+1)}$$

- **ETS(AMM)**: Аддитивная ошибка, мультипликативный тренд, мультипликативная сезонность

  $$l_t \cdot b_t^h \cdot s_{t+h-m(k+1)}$$

- **ETS(AAdA)**: Аддитивная ошибка, затухающий тренд, аддитивная сезонность

Первая буква (Error) не влияет на формулу точечного прогноза.
Ошибка $\varepsilon_t$ критически важна в обновлении состояний и расчета доверительных интервалов
**Ошибка M предельно редко**

# Аддитивные компоненты

**Аддитивная ошибка**
Шум не зависит от уровня ряда. Подходит, если волатильность постоянна

**Аддитивный тренд**
Предполагает, что изменения уровня со временем происходят линейно.
Это значит, что к уровню каждый шаг прибавляется (или вычитается) постоянная величина. Тренд растёт или падает с постоянной скоростью

**Аддитивная сезонность**
Означает, что сезонные колебания добавляются к уровню ряда, и их амплитуда остаётся постоянной независимо от роста или падения уровня

# Мультипликативные компоненты

## Мультипликативная ошибка

Шум пропорционален уровню. Подходит, если разброс данных растет вместе с ростом значений

## Мультипликативный тренд

Предполагает, что тренд изменяется пропорционально уровню, а не в виде постоянной добавляемой величины. То есть, при мультипликативном тренде каждое последующее изменение определяется текущим уровнем.

## Мультипликативная сезонность

Амплитуда колебаний изменяется пропорционально уровню ряда. То есть сезонные эффекты становятся сильнее, когда уровень данных высокий, и слабее, когда уровень низкий. При мультипликативной сезонности сезонные колебания умножаются на значение ряда, и это полезно, когда сезонные колебания растут или убывают вместе с трендом.

# ExponentialSmoothing в statsmodels

Может возвращать NaN при прогнозировании. Причины:

- **Недостаток данных для сезонности**
  Например, для seasonal_periods=12 необходимо как минимум 2-3 полных сезона данных, иначе модель может не обучиться
- **Неподходящий параметр модели**
  Временной ряд может не подходить для аддитивной или мультипликативной модели, и указание неподходящего параметра может вызвать ошибку. Например, если у вас есть нулевые или отрицательные значения, seasonal='mul' вызовет ошибку, так как мультипликативная сезонность не может корректно обработать такие данные.
- **Проблемы с параметрами оптимизации**

**Как починить**

1. Проверьте, работает ли модель, если исключить тренд или сезонность
2. Попробуйте использовать аддитивные компоненты
3. Обработайте возможные аномалии в данных

# Для каких рядов использовать ETS

- **Сезонные ряды**
- **Долгосрочные тренды (ближе к балансу долгосрока и краткосрока)**
- **Шумы и нестабильность**
- **Нет строгих предпосылок о распределении**
  ETS не требует строгих предпосылок о распределении ошибок (например, нормальное распределение), что делает его подходящим для более широкого круга временных рядов.
- **Отсутствие автокорреляции**

# Где нет смысла доставать ETS

**Сложные временные ряды**

Если временной ряд имеет сложную структуру, включая несколько уровней сезонности или структурные изменения (например, резкие скачки или переходы)

**Мультисезонность**

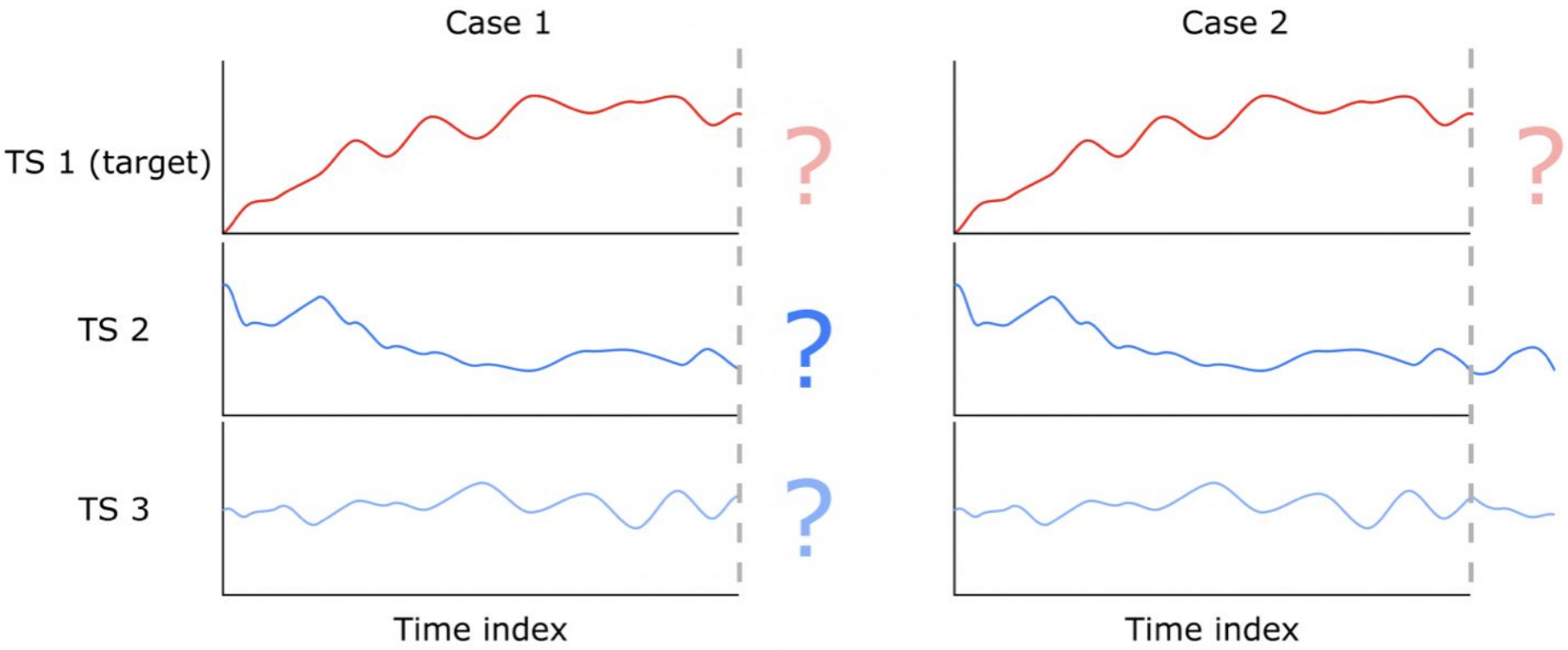
**Данные с сильной автокорреляцией**

Если временной ряд имеет высокую автокорреляцию, модели, такие как ARIMA или SARIMA, могут быть более подходящими, так как они специально разработаны для обработки зависимостей между временными точками

**Большое количество аномалий**

**Данные с нестабильным уровнем**

# Многомерные ряды


Case 1
Case 2
TS 1 (target)
TS 2
TS 3
?
?
?
?
Time index
Time index

# VAR

Обобщение идеи авторегрессии для нескольких временных рядов

Несколько временных рядов могут включаться в классические модели прогнозирования, такую как, например, ARIMA (VARMA)

задача multivariate time series forecasting

$$Y_{1,t} = c_1 + a_{11}Y_{1,t-1} + a_{12}Y_{2,t-1} + \epsilon_1$$
$$Y_{2,t} = c_2 + a_{21}Y_{1,t-1} + a_{22}Y_{2,t-1} + \epsilon_2$$

Обязательное условие **стационарности** всех рядов

Если вы возьмете два абсолютно независимых, но растущих ряда, и построите между ними регрессию, модель скажет: «У них сильная связь»

Приводя к стационарным через дифференцирование **проблема**:

Мы знаем, как ряд меняется, но забываем, на каком он уровне. Мы теряем информацию о долгосрочном равновесии

# Коинтеграция

Пьяница идет по улице случайным образом (шатается влево-вправо). Его путь — нестационарный ряд (случайное блуждание).

Собака тоже бегает случайно. Её путь — тоже нестационарный ряд.

Но! Они связаны поводком

**Суть коинтеграции:**

- Сам по себе путь пьяницы непредсказуем.
- Сам по себе путь собаки непредсказуем.
- НО расстояние между ними (длина поводка) ограничено и стабильно.
- Это расстояние — стационарный ряд

**Если ряды НЕ коинтегрированы**

Мы просто берем разности и строим VAR

**Если ряды Коинтегрированы и имеют единый порядок приведения к стационарному**

Используется VECM

# Тест Йохансена

Тест проверяет ранг матрицы долгосрочных связей (П-матрица) в системе VAR

$$\Delta Y_t = \Pi Y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta Y_{t-i} + \varepsilon_t$$

П — это матрица, которая содержит информацию о долгосрочных связях

**r = 0**
нет коинтеграции; VAR на разностях

**0 < r < k**
есть r коинтегрирующих векторов из k в системе

**r = k**
все ряды стационарны; VAR

# Тест Йохансена

**След-тест**

Сколько всего есть коинтегрирующих векторов?

$$-T\sum_{i=r+1}^{n}\ln(1-\lambda_i)$$

λ - собственные значения
r - число коинтегрирующих соотношений
T - количество наблюдений

**Дано**

T = 100 наблюдений

n = 3 переменных (нефть, газ, уголь)

Собственные значения: $\lambda_1 = 0.45$, $\lambda_2 = 0.20$, $\lambda_3 = 0.05$

**Проверяем $H_0$:** r = 0 (нет коинтеграции)

$-100 \cdot [\ln(1-0.45)+\ln(1-0.20)+\ln(1-0.05)]=$
$-100 \cdot [-0.598-0.223-0.051]=87.2$

**Проверяем $H_0$:** r ≤ 1 (не более 1 вектора)

$-100 \cdot [\ln(1-0.20)+\ln(1-0.05)]=$
$-100 \cdot [-0.223-0.051]=27.4$

Результат сравнивается с критическими значениями полученных стохастическими симуляциями

**Если статистика > критического значения**:

ОТВЕРГАЕМ $H_0$

**Если статистика < критического значения**:

НЕ ОТВЕРГАЕМ $H_0$

# Тест Йохансена

**Дано**

T = 100 наблюдений

n = 3 переменных (нефть, газ, уголь)

Собственные значения: $\lambda_1 = 0.45$, $\lambda_2 = 0.20$, $\lambda_3 = 0.05$

**Проверяем $H_0$:** r = 0 (нет коинтеграции)

$-100 \cdot \ln(1-0.45) = -100 \cdot (-0.598) = 59.8$

**Проверяем $H_0$:** $r \leq 1$ (не более 1 вектора)

$-100 \cdot \ln(1-0.20) = 22.3$

Результат сравнивается с критическими значениями полученных стохастическими симуляциями

**ЗНАЧЕНИЯ ДЛЯ ДВУХ ТЕСТОВ ОТЛИЧАЮТСЯ**

**Тест максимального собственного числа**

Есть ли ещё один вектор сверх найденных?

$$-T \ln(1 - \lambda_{r+1})$$

# VECM - Vector Error Correction Model

VECM = VAR на разностях + механизм коррекции ошибки

$$\Delta Y_t = \alpha_1 + \sum_{i=1}^{p-1} \gamma_{11,i} \Delta Y_{t-i} + \sum_{i=1}^{p-1} \gamma_{12,i} \Delta X_{t-i} + \lambda_1 \cdot \underbrace{(Y_{t-1} - \beta_0 - \beta_1 X_{t-1})}_{\text{Error Correction Term}} + \varepsilon_{1t}$$

$$\Delta X_t = \alpha_2 + \sum_{i=1}^{p-1} \gamma_{21,i} \Delta Y_{t-i} + \sum_{i=1}^{p-1} \gamma_{22,i} \Delta X_{t-i} + \lambda_2 \cdot \underbrace{(Y_{t-1} - \beta_0 - \beta_1 X_{t-1})}_{\text{Error Correction Term}} + \varepsilon_{2t}$$

Коэффициенты скорости возврата к равновесию умножаются на отклонение от него

**Если $\lambda_1 < 0$ и статистически значим**

- Когда Y выше равновесного уровня, в следующем периоде Y будет снижаться
- Отрицательный λ = стабилизирующая сила ("поводок")

**Если $\lambda \approx 0$**

- Ряд не реагирует на отклонение от равновесия
- Возможно, коинтеграция ложная или слабая

# VECM - Vector Error Correction Model

VECM не учится итеративно с помощью градиентного спуска.
Он использует статистический метод Максимального Правдоподобия

**Шаг 1: Поиск равновесия**

Модель перебирает возможные комбинации рядов и ищет такую линейную связь β, чтобы остаток был максимально стационарным (дисперсия ошибки минимальна)

Технически: Решается задача на собственные значения ковариационной матрицы остатков

**Шаг 2: Поиск динамики**

Методом наименьших квадратов подбирает насколько сильно отклонение влияет на возврат, как прошлые изменения влияют на текущие

**Шаг 3: Максимизация правдоподобия**

Параметры подбираются так, чтобы вероятность наблюдения ваших реальных данных при данной модели была максимальной

# TCN

# Temporal Convolutional Network (TCN)

Архитектура нейросети для работы с последовательностями, основанная на свёртках, а не на рекуррентных связях

**Одномерная (dilated) свертка**

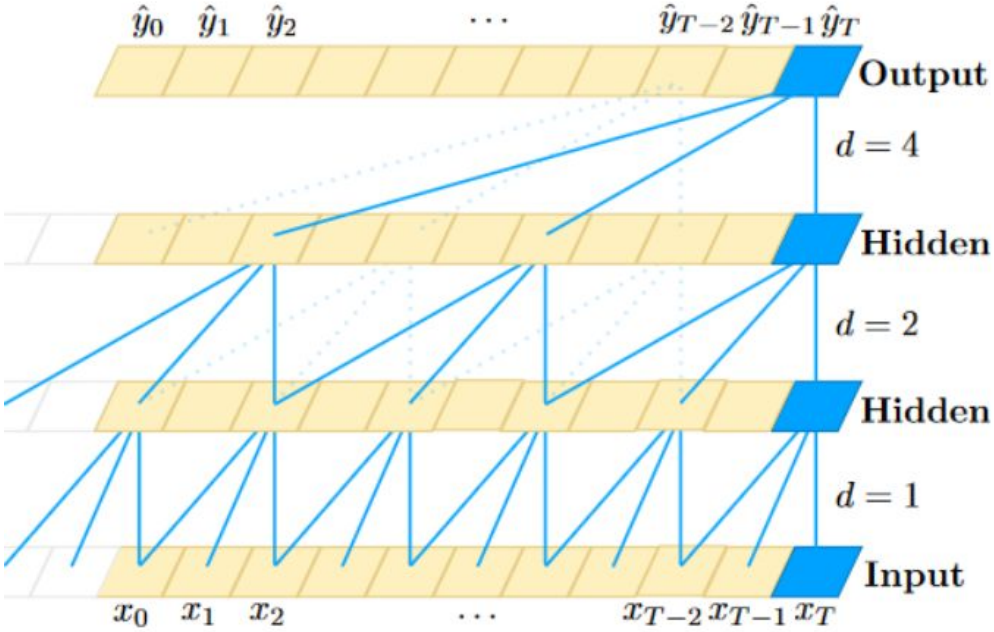
вход (x_0,...,x_T), выход y_t = f(x_0,...,x_t)

$$F(s) = (\mathbf{x} *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i}$$

Отличие от привычной операции одномерной свертки в том, что вместо индекса входа **s-i** берется **s-d*i**, то есть свертка берет предыдущие входы с некоторым "шагом", **пропуская ряд шагов**
d - фактор расширения (dilation factor), шаг пропуска
k - размер фильтра, сколько шагов берем

# Temporal Convolutional Network (TCN)

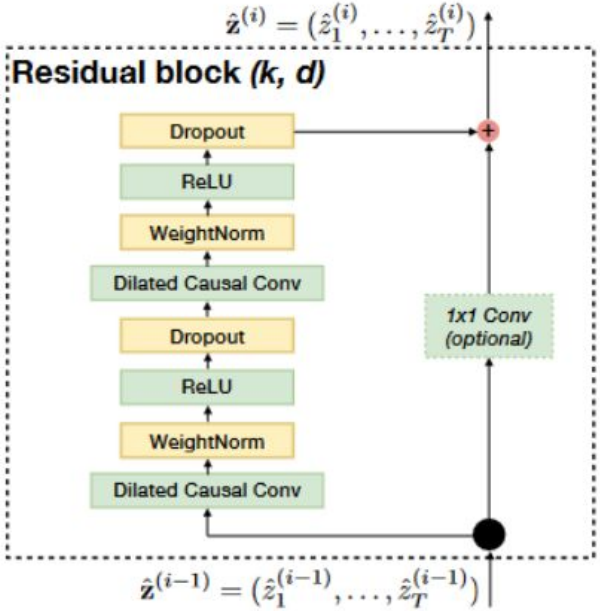
Окно свертки увеличивается экспоненциально от слоя к слою

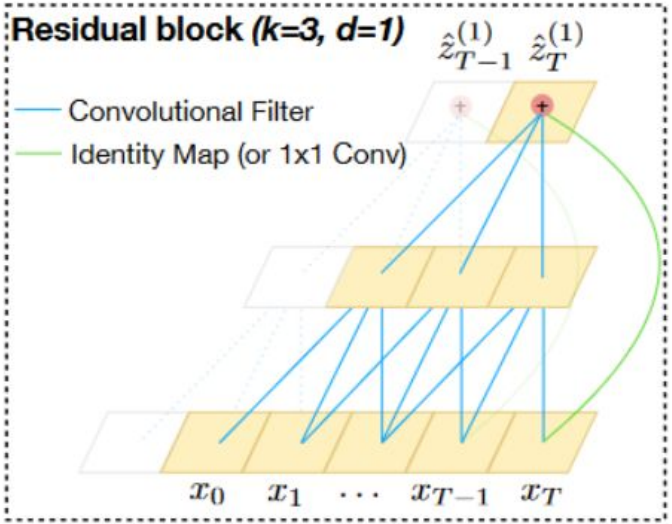



**Мелкие слои (d=1, 2):**
улавливают быстрые изменения

**Глубокие слои (d=64, 128):**
видят глобальные, долгосрочные тренды

# Temporal Convolutional Network (TCN)




Обходной путь skip connections применяет применяется свертку 1×1 (просто линейное преобразование)

Суммируются результаты основного пути и обходного

Благодаря этому **можно делать сотни слоев**, идет борьба с затуханием градиентов

# Temporal Convolutional Network (TCN)

## Плюсы

- Параллелизм
- Длинная память
- Стабильные градиенты
- Гибкая длина входа
- Сходятся быстрее чем RNN

## Минусы

- Много памяти на обучении
- При экстремально длинных зависимостях проигрывают трансформерам
- Слишком больше d теряет детали
- Требуют тщательной настройки
- Не подходят для очень маленьких датасетов: берите LSTM

# N-BEATS/N-HiTS